

Assignment -II Solution

- Subject: Database Management System

1) Given relation schema as below

Employee(emp_id, name, address, telephone, salary, age)

Works_on (emp_id, project_id, join_date)

Project(project_id, project_name, city,duration, budget)

Write the SQL commands for the following.

1. Insert new record in project relation.

```
INSERT INTO Project  
VALUES (102, 'DBMS project', 'kathmandu', 2, 55000);
```

2. Find the name of employees with the name of project they work on.

```
SELECT Employee.name, Project.project_name  
FROM Employee, Works_on, Project  
WHERE Employee.emp_id = Works_on.emp_id  
AND Works_on.project_id = Project.project_id;
```

3. Find the name and city of that project on which salary of employee is greater than or equal to 20000.

```
SELECT Project.project_name, Project.city  
FROM Employee, Works_on, Project  
WHERE Employee.emp_id = Works_on.emp_id  
AND Works_on.project_id = Project.project_id  
AND Employee.salary >= 20000;
```

4. List the name of employees whose name starts with “m” and ends with “a”

```
SELECT name  
FROM Employee  
WHERE name LIKE 'm%a';
```

5. Find the employee name and project name of those employees who living in address Pokhara.

```
SELECT Employee.name, Project.project_name  
FROM Employee, Works_on, Project  
WHERE Employee.emp_id = Works_on.emp_id  
AND Works_on.project_id = Project.project_id  
AND Employee.address = 'Pokhara';
```

6. List name of employee whose age is greater than average age of all employees.

```
SELECT name
```

```
FROM Employee
WHERE age > (SELECT AVG(age) FROM Employee);
```

7. **List employee id of all employees whose age is greater than minimum budget of all projects.**

```
SELECT emp_id
FROM Employee
WHERE age > (SELECT MIN(budget) FROM Project);
```

8. **List employee id of all employees who joint project on “05/01/2015”**

```
SELECT emp_id
FROM Works_on
WHERE join_date = '2015-05-01';
```

9. **List the name of employees whose name starts with N or with K**

```
SELECT name
FROM Employee
WHERE name LIKE 'N%' OR name LIKE 'K%';
```

10. **Display the project id with maximum budget.**

```
SELECT project_id
FROM Project
WHERE budget = (SELECT MAX(budget) FROM Project);
```

- 2) Given Relational Schema as below.

```
Sailors (sid, sname, age, rating)
Boats (bid, bname, color)
Reserves (sid, bid, day)
Write the SQL for the following.
```

Note: For your convenience, rows in the above relation are inserted as needed, facilitating easier demonstration of the output. However, during exams, it is not mandatory to insert all rows unless prompted by specific questions.

The output is generated to verify whether the query conforms to the question's requirements, ensuring its relevance and accuracy. But it is not required in exam.

Sailors

sid	sname	age	rating
22	Dustin	45	7
29	Brutus	33	1
31	Lubber	55	8
32	Andy	25	8
58	Rusty	35	10
64	Horatio	35	7
71	Zorba	16	10
74	Horatio	40	9
85	Art	25	3
95	Bob	63	3

Boats

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Reserves

sid	bid	day
22	101	1998-10-10
22	102	1998-10-10
22	103	1998-10-08
22	104	1998-10-07
31	102	1998-11-10
31	103	1998-11-06
31	104	1998-11-12
64	101	1998-09-05
64	102	1998-09-08
74	103	1998-09-08

1. Find the sid of the sailor who have reserved red or green boat.

```
SELECT distinct Sailors.sid
```

```

FROM Sailors, Boats, Reserves
WHERE Sailors.sid = Reserves.sid
AND Reserves.bid = Boats.bid
AND (Boats.color = 'red' OR Boats.color = 'green');

```

Output

sid
22
31
64
74

2. Find sid's of sailors who've reserved a red and a green boat.

```

SELECT Sailors.sid
FROM Sailors, Boats, Reserves
WHERE Sailors.sid = Reserves.sid
AND Reserves.bid = Boats.bid
AND Boats.color = 'red'
INTERSECT
SELECT Sailors.sid
FROM Sailors, Boats, Reserves
WHERE Sailors.sid = Reserves.sid
AND Reserves.bid = Boats.bid
AND Boats.color = 'green';

```

Output

sid
22
31

3. Find names of sailors who have reserved boat id 103:

```

SELECT Sailors.sname
FROM Sailors, Reserves
WHERE Sailors.sid = Reserves.sid
AND Reserves.bid = 103;

```

sname
Dustin
Lubber
Horatio

4. Find sailors who've reserved all boats

```
SELECT Sailors. sname
FROM Sailors
WHERE NOT EXISTS
(SELECT Boats.bid FROM Boats
WHERE NOT EXISTS
( SELECT Reserves.bid FROM Reserves WHERE Reserves.bid=Boats.bid AND
Reserves.sid=Sailors.sid));
```

sname
Dustin

This SQL query retrieves the names of sailors who have reserved all boats. Here's an explanation of each part of the query:

- ✓ **SELECT Sailors.sname:** This is the main query that selects the sailor names from the Sailors table.
- ✓ **FROM Sailors:** Specifies that the data is being selected from the Sailors table.
- ✓ **WHERE NOT EXISTS (SELECT Boats.bid FROM Boats WHERE NOT EXISTS (SELECT Reserves.bid FROM Reserves WHERE Reserves.bid=Boats.bid AND Reserves.sid=Sailors.sid)):** This is a subquery that filters the sailors who have reserved all boats.
- ✓ The innermost subquery (**SELECT Reserves.bid FROM Reserves WHERE Reserves.bid=Boats.bid AND Reserves.sid=Sailors.sid**) checks for each sailor (Sailors.sid) whether there exists a reservation for a boat (Reserves.bid) that matches both the sailor and the boat.
- ✓ The middle subquery (**SELECT Boats.bid FROM Boats WHERE NOT EXISTS (SELECT Reserves.bid FROM Reserves WHERE Reserves.bid=Boats.bid AND Reserves.sid=Sailors.sid)**) checks for each boat (Boats.bid) whether there does not exist a reservation by the current sailor for that boat.
- ✓ The outer query **WHERE NOT EXISTS (SELECT Boats.bid FROM Boats WHERE NOT EXISTS ...)** checks if there are no boats for which a reservation does not exist for the current sailor.
- ✓ Finally, the outermost WHERE clause ensures that only sailors who do not meet the condition of not having reserved all boats are selected.

Overall, the query ensures that it selects sailors who have made reservations for all available boats.

5. Find the name and age of youngest sailor

```
SELECT Sailors.sname, Sailors.age
FROM Sailors
WHERE Sailors.age = (SELECT MIN(Sailors.age)
FROM Sailors Sailors );
```

sname	age
Zorba	16

6. Find name and age of the oldest sailor(s) with rating >7

```
SELECT Sailors.sname, Sailors.age
FROM Sailors
WHERE Sailors.rating > 7
AND Sailors.age = (
    SELECT MAX(Sailors.age)
    FROM Sailors
    WHERE Sailors.rating > 7
);
```

sname	age
Lubber	55

7. Find the age of the youngest sailor for each rating level

```
SELECT rating, MIN(Age) as youngest_age
FROM Sailors
GROUP BY rating;
```

rating	youngest_age
1	33
3	25
7	35
8	25
9	40
10	16

8. Find the age of the youngest sailor with age >= 18 for each rating level with at least 2 such sailors

```
SELECT rating, MIN(Age)
FROM Sailors
```

```

WHERE age >= 18
GROUP BY rating
HAVING COUNT(*) >= 2;

```

rating	MIN(Age)
3	25
7	35
8	25

9. Find the average age for each rating, and order results in ascending order on avg.

```

SELECT rating, AVG(age) AS avg_age
FROM Sailors
GROUP BY rating
ORDER BY avg_age ASC;

```

rating	avg_age
10	25.5000
1	33.0000
9	40.0000
7	40.0000
8	40.0000
3	44.0000

- 3) Consider the relational database of figure given below, where primary keys are underlined.
Given an expression in SQL for each of the following queries.

```

Employee(employee_name, street, city)
Works(employee_name, company_name, salary)
Company(company_name, city)

```

Manages(employee_name,manager_name)

Note: For your convenience, rows in the above relation are inserted as needed, facilitating easier demonstration of the output. However, during exams, it is not mandatory to insert all rows unless prompted by specific questions.

The output is generated to verify whether the query conforms to the question's requirements, ensuring its relevance and accuracy. But it is not required in exam.

Employee

employee_name	street	city
John	123 Main St	New York
Alice	456 Elm St	New York
Bob	123 Maple Avenue	Springfield
Mary	123 Main st	New York
David	222 Cedar St	Chicago

Works

employee_name	company_name	salary
John	First Bank Corporation	15000.00
Alice	First Bank Corporation	12000.00
Bob	Small Bank Corporation	10000.00
Mary	Small Bank Corporation	18000.00
David	Big Corporation	20000.00

company

company_name	city
First Bank Corporation	New York
Small Bank Corporation	Los Angeles
Big Corporation	Chicago

Manages

employee_name	manager_name
Alice	John
Bob	Alice
Mary	John
David	Mary

- i. Find the names of all employees who work for the First Bank Corporation.

```
SELECT employee_name
FROM Works
WHERE company_name = 'First Bank Corporation';
```

Output

employee_name
John
Alice

- ii. Find the names of all employees who live in the same city and on the same street as do their managers.

```
SELECT E1.employee_name
FROM Employee AS E1, Employee AS E2, Manages AS M
WHERE E1.employee_name = M.employee_name
AND E2.employee_name = M.manager_name
AND E1.street = E2.street
AND E1.city = E2.city;
```

Output

employee_name
Mary

✓ **SELECT clause:** Finally, the SELECT clause specifies what data to retrieve. In this case, it selects the employee_name from E1, which represents the name of the employees who meet all the conditions specified in the WHERE clause.

- ✓ **FROM clause:** The query starts by selecting from three tables: Employee (twice, aliased as E1 and E2) and Manages (aliased as M). This is the source of data for the query.
- ✓ **WHERE clause:** The conditions specified in the WHERE clause filter the results. Here's what each condition does:
 - **E1.employee_name = M.employee_name:** This condition ensures that the employee in E1 is the same as the employee in the Manages table, meaning that E1 is an employee who is managed by someone.
 - **E2.employee_name = M.manager_name:** This condition ensures that E2 represents the manager of the employee in E1.
 - **E1.street = E2.street:** This condition checks if both the employee (E1) and their manager (E2) live on the same street.
 - **E1.city = E2.city:** This condition checks if both the employee (E1) and their manager (E2) live in the same city.

By combining these elements, the query retrieves the names of employees who live in the same city and on the same street as their managers. This is achieved by joining the Employee table with itself (via the E1 and E2 aliases) based on the relationships defined in the Manages table and filtering the results based on the specified conditions.

iii. Find the names, street address, and cities of residence of all employees who work for First Bank Corporation and earn more than \$10,000 per annum

```
SELECT Employee.employee_name, Employee.street, Employee.city
FROM Employee, Works
WHERE Employee.employee_name = Works.employee_name
AND Works.company_name = 'First Bank Corporation'
AND Works.salary > 10000;
```

Output

employee_name	street	city
John	123 Main St	New York
Alice	456 Elm St	New York

iv. Find the names of all employees who earn more than every employee of Small Bank Corporation

```
SELECT employee_name
FROM Works
WHERE salary > ALL
(
```

```

SELECT salary
FROM Works
WHERE company_name = 'Small Bank Corporation'
);

```

Output

employee_name
David

- v. Find those companies whose employees earn a higher salary, on average, than the average salary at First Bank Corporation.

```

SELECT company_name
FROM Works
GROUP BY company_name
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM Works
    WHERE company_name = 'First Bank Corporation'
);

```

Output

company_name
Big Corporation
Small Bank Corporation

- vi. Find the names of all employees in this database who live in the same city as the company for which they work

```

SELECT Employee.employee_name
FROM Employee, Works, Company
WHERE Employee.employee_name = Works.employee_name
AND Works.company_name = Company.company_name
AND Employee.city = Company.city;

```

employee_name
John
Alice
David

- vii. modify the databases so that John now lives in Chicago

```
UPDATE Employee
SET city = 'Chicago'
WHERE employee_name = 'John';
```

Now ,Employee table becomes

employee_name	street	city
John	123 Main St	Chicago
Alice	456 Elm St	New York
Bob	123 Maple Avenue	Springfield
Mary	123 Main st	New York
David	222 Cedar St	Chicago

- viii. Give all employees of First Bank Corporation a 10 percent raise

```
UPDATE Works
SET salary = salary * 1.1
WHERE company_name = 'First Bank Corporation';
```

Now works table looks like,

employee_name	company_name	salary
John	First Bank Corporation	16500.00
Alice	First Bank Corporation	13200.00
Bob	Small Bank Corporation	10000.00
Mary	Small Bank Corporation	18000.00
David	Big Corporation	20000.00

- ix. Give salary of all managers of First Bank Corporation a 10 percent raise.

```
UPDATE Works
SET salary = salary * 1.1
WHERE employee_name IN (SELECT manager_name FROM Manages)
AND company_name = 'First Bank Corporation';
```

Note that John and Alice both are managers of First Bank corporation

employee_name	company_name	salary
John	First Bank Corporation	18150.00
Alice	First Bank Corporation	14520.00
Bob	Small Bank Corporation	10000.00
Mary	Small Bank Corporation	18000.00
David	Big Corporation	20000.00

- x. Delete all in the work relation for employees of small bank corporation.

```
DELETE FROM Works
WHERE company_name = 'small bank corporation';
```

Now ,works relation becomes

employee_name	company_name	salary
John	First Bank Corporation	18150.00
Alice	First Bank Corporation	14520.00
David	Big Corporation	20000.00

- xi. Find all employees who earn more than the average salary of all employees of their company

```
SELECT employee_name
FROM works a
WHERE salary > (SELECT AVG(salary)
FROM works b
WHERE a.company_name = b.company_name);
```

Output

employee_name
John

- 4) Consider the following schema

```
Branch(bname, bcity, assets)
Customer(cname, street, ccity)
```

```
Depositor (cname, account#)
Account(bname, account#, balance)
Loan(bname, loan#, amount)
Borrower(cname, loan#)
```

Write SQL statement for the following

1. **Find the name of all branch in account relation.**

```
SELECT DISTINCT bname
FROM Account;
```

2. **Finds the names of all branches that have assets greater than at least one branch located in Burnaby.**

```
SELECT bname
FROM Branch
WHERE assets > some (SELECT assets FROM Branch WHERE bcity = 'Burnaby');
```

3. **Find all customers whose street includes the substring “Main”.**

```
SELECT cname
FROM Customer
WHERE street LIKE '%Main%';
```

4. **Finds all customers who have a loan and an account at the SFU branch.**

```
SELECT DISTINCT Borrower.cname
FROM Borrower, Loan
WHERE Loan.`loan#` = Borrower.`loan#`
AND Loan.bname = 'SFU'
INTERSECT
SELECT DISTINCT Depositor.cname
FROM Depositor, Account
WHERE Depositor.`account#` = Account.`account#`
AND Account.bname = 'SFU';
```

5. **Find the name and loan number of all customers who have a loan at SFU branch.**

```
SELECT DISTINCT Borrower.cname, Borrower.`loan#`
FROM Borrower, Loan
WHERE Borrower.`loan#` = Loan.`loan#`
AND Loan.bname = 'SFU';
```

6. **Finding all customers who have a loan but not an account at the SFU branch**

```
SELECT DISTINCT Borrower.cname
FROM Borrower, Loan
WHERE Loan.`loan#` = Borrower.`loan#`
AND Loan.bname = 'SFU'
EXCEPT
```

```

SELECT DISTINCT Depositor.cname
FROM Depositor, Account
WHERE Depositor.`account#` = Account.`account#`
AND Account.bname = 'SFU';

```

7. Find the branches whose assets are greater than some branch in Pokhara.

```

SELECT bname
FROM Branch
WHERE assets > SOME(
    SELECT assets
    FROM branch
    WHERE bcity='Pokhara'
);

```

5) Consider the following relational schema

```

branch (branch-name, branch-city, assets)
customer (customer-name, customer-street, customer-city)
account (account-number, branch-name, balance)
loan (loan-number, branch-name, amount)
depositor (customer-name, account-number)
borrower (customer-name, loan-number)

```

Write relational algebraic expressions for the following

1. Find all loans of over \$1200

$(\sigma_{\text{amount} > 1200}(\text{loan}))$

2. Find the loan number for each loan of an amount greater than \$1200

$\Pi_{\text{loan-number}} (\sigma_{\text{amount} > 1200} (\text{loan}))$

3. Find the names of all customers who have a loan, an account, or both, from the bank.

$\Pi_{\text{customer-name}} (\text{borrower}) \cup \Pi_{\text{customer-name}} (\text{depositor})$

4. Find the names of all customers who have a loan and an account at bank.

$\Pi_{\text{customer-name}} (\text{borrower}) \cap \Pi_{\text{customer-name}} (\text{depositor})$

5. Find the names of all customers who have a loan at the Perryridge branch.

$\Pi_{\text{customer-name}} (\sigma_{\text{branch-name} = \text{"Perryridge"}} (\text{borrower} \bowtie \text{loan}))$

6. Find the names of all customers who have a loan at the Perryridge branch but do not have an account at any branch of the bank.

$\Pi_{\text{customer-name}} (\sigma_{\text{branch-name} = \text{"Perryridge"}} (\text{borrower} \bowtie \text{loan})) - \Pi_{\text{customer-name}} (\text{depositor})$

7. Find the names of all customers who have a loan at the Perryridge branch.

$$\Pi_{\text{customer-name}}(\sigma_{\text{branch-name} = \text{"Perryridge"}}(\text{borrower} \bowtie \text{loan}))$$

8. Find the largest account balance.

$$\mathcal{G}_{\max(\text{balance})}(\text{account})$$

9. Find all customers who have an account from at least the “Downtown” and the Uptown” branches.

$$\Pi_{\text{customer-name}}(\sigma_{\text{branch-name} = \text{"Downtown"}}(\text{depositor} \bowtie \text{account})) \cap \\ \Pi_{\text{customer-name}}(\sigma_{\text{branch-name} = \text{"Uptown"}}(\text{depositor} \bowtie \text{account}))$$

10. Find all customers who have an account at all branches located in Brooklyn city.

$$\Pi_{\text{customer-name}, \text{branch-name}}(\text{depositor} \bowtie \text{account}) \\ \div \Pi_{\text{branch-name}}(\sigma_{\text{branch-city} = \text{"Brooklyn"}}(\text{branch}))$$

11. Delete all account records in the Perryridge branch.

$$\text{account} \leftarrow \text{account} - \sigma_{\text{branch-name} = \text{"Perryridge"}}(\text{account})$$

12. Delete all loan records with amount in the range of 0 to 50.

$$\text{loan} \leftarrow \text{loan} - \sigma_{\text{amount} \geq 0 \text{ and } \text{amount} \leq 50}(\text{loan})$$

13. Delete all accounts at branches located in Needham.

$$r_1 \leftarrow \Pi_{\text{branch-name}, \text{account-number}, \text{balance}}(\sigma_{\text{branch-city} = \text{"Needham"}}(\text{account} \bowtie \text{branch})) \\ r_2 \leftarrow \Pi_{\text{customer-name}, \text{account-number}}(r_1 \bowtie \text{depositor}) \\ \text{account} \leftarrow \text{account} - r_1 \\ \text{depositor} \leftarrow \text{depositor} - r_2$$

14. Insert information in the database specifying that Smith has \$1200 in account A-973 at the Perryridge branch.

$$\text{account} \leftarrow \text{account} \cup \{(\text{"Perryridge"}, \text{A-973}, 1200)\} \\ \text{depositor} \leftarrow \text{depositor} \cup \{(\text{"Smith"}, \text{A-973})\}$$

15. Update interest payments by increasing all balances by 5 percent.

$$\text{account} \leftarrow \Pi_{\text{account-number}, \text{branch-name}, \text{balance} * 1.05}(\text{account})$$

16. Update all accounts with balances over \$10,000 6 percent interest and pay all other 5 percent.

$$\text{account} \leftarrow \Pi_{\text{account-number}, \text{branch-name}, \text{balance} * 1.06}(\sigma_{\text{balance} > 10000}(\text{account})) \\ \cup \Pi_{\text{account-number}, \text{branch-name}, \text{balance} * 1.05}(\sigma_{\text{balance} \leq 10000}(\text{account}))$$

17. Create the view (named all-customer) consisting of branches and their customers.

create view all-customer as

$$\Pi_{\text{branch-name, customer-name}} (\text{depositor} \bowtie \text{account}) \\ \cup \Pi_{\text{branch-name, customer-name}} (\text{borrower} \bowtie \text{loan})$$

6) Consider the following relational database, where primary keys are underlined.

employee(person_name,street,city)

works(person_name,company_name,salary)

company(company_name,city)

manages(person_name,manager_name)

Given an expression in the relational algebra to express each of the following queries.

i) Find the name of all employees who work for first bank corporation.

$$\Pi_{\text{person_name}} (\sigma_{\text{company_name}=\text{"first bank corporation"}}(\text{works}))$$

ii) Find the name and cities of residence of all employees who work for first bank corporation.

$$\Pi_{\text{person_name,city}} (\sigma_{\text{company_name}=\text{"first bank corporation"}}(\text{employee} \bowtie \text{works}))$$

iii) Find the name, street address and city of residence of all employees who work for first bank corporation and earn more than \$10000 per annum.

$$\Pi_{\text{person_name,street,city}} (\sigma_{\text{company_name}=\text{"first bank corporation"} \wedge \text{salary} > 10000}(\text{employee} \bowtie \text{works}))$$

(Here we assume attribute named salary represent Annual Salary)

iv) Find the name of all employees in this database who lives in same city as the company for which they work.

$$\Pi_{\text{person_name}}(\text{employee} \bowtie \text{works} \bowtie \text{company})$$

v) Find the name of all employees who live in the same city and on the same street as do their managers.

$$\Pi_{\text{person_name}} ((\text{employee} \bowtie \text{manages}) \bowtie_{(\text{manages.manager_name}=\text{employee2.person_name} \wedge \text{employee.street}=\text{employee2.street} \wedge \text{employee.city}=\text{employee2.city})} (\rho_{\text{employee2}} (\text{employee})))$$

vi) Find the name of all employees in this database who do not work for first bank corporation.

$$\Pi_{\text{person_name}} (\sigma_{\text{company_name} \neq \text{"first bank corporation"}}(\text{works}))$$

vii) Display the name of employee whose name begin from S.

viii) Find the average salary of employee.

$$G_{\text{avg(salary)}} (\text{works})$$

ix) Find the average salary of employee company wise.

$$\text{company_name} \xrightarrow{G} \text{avg}(\text{salary}) (\text{works})$$

x) Find the name of all employees who earn more than their managers.

$$\Pi \text{ person_name } ((\text{works} \bowtie \text{manages}) \bowtie_{\text{manages.manager_name}=\text{e2.person_name} \wedge \text{works.salary} > \text{e2.salary}} (\rho_{\text{e2}}(\text{works})))$$

In SQL this will work

```
SELECT works.person_name
FROM works NATURAL JOIN manages
JOIN works AS e2 ON manages.manager_name = e2.person_name
AND works.salary > e2.salary;
```

xi) Find the name of employees who earn more than top earner at “NBL company” in the database.

$$\text{topearner} \leftarrow \xrightarrow{G} \max(\text{salary}) (\sigma_{\text{company_name}=\text{"NBL company"}}(\text{works}))$$

$$\text{result} \leftarrow \Pi \text{ person_name} (\sigma_{\text{salary} > \text{topearner}}(\text{works}))$$

xii) Find the name of all employee who live in “Lalitpur” and salary is less than 50000

$$\Pi \text{ person_name} (\sigma_{\text{city}=\text{"Lalitpur"} \wedge \text{salary} < 50000} (\text{Employee} \bowtie \text{Works}))$$

xiii) Assume that companies may be located in several cities. Find all companies located in every city in which small bank corporation is located.

$$\Pi \text{ company_name, city} (\text{company}) \div \Pi \text{ city} (\sigma_{\text{company_name}=\text{"small bank corporation"}}(\text{company}))$$

xiv) List the name and city of employee who work in “Pokhara” and have salary greater than Rs.50000

$$\Pi \text{ Employee.person_name, Employee.city } (\sigma_{\text{Company.city}=\text{"Pokhara"} \wedge \text{salary} > 50000} (\text{Employee} \bowtie_{\text{Employee.person_name}=\text{works.person_name}} (\text{works} \bowtie \text{company})))$$

xv) Delete all employee who come from “Chitwan”.

$$\text{Employee} \leftarrow \text{Employee} - \sigma_{\text{city}=\text{"Chitwan"}}(\text{Employee})$$

xvi) Increase salary of all employee by 15%.

$$\text{Works} \leftarrow \Pi \text{ person_name, company_name, salary} * 1.15 (\text{Works})$$

xvii) Insert new records in employee relation.

$$\text{employee} \leftarrow \text{employee} \cup \{ \text{"Ram"}, \text{"patan"}, \text{"Lalitpur"} \}$$

xviii) Create view for which employee earns 20000\$ or more

Create view high_earning_view

$\sigma_{\text{salary} \geq 20000} (\Pi_{\text{person_name}, \text{street}, \text{city}, \text{company_name}, \text{salary}} (\text{employee} \bowtie \text{works}))$

xix) Modify the database so that Ramesh now lives in Kathmandu

$\text{employee} \leftarrow \Pi_{\text{person_name}, \text{street}, \text{city}} (\sigma_{\text{person_name} = \text{'Ramesh'}} (\text{employee}))$
 $\cup (\text{employee} - \sigma_{\text{person_name} = \text{'Ramesh'}} (\text{Employee}))$

xx) Give all salary of employee of “ABC company” 18.5% rise

$\text{works} \leftarrow \Pi_{\text{person_name}, \text{company_name}, \text{salary}} (\sigma_{\text{company_name} = \text{'ABC company'}} (\text{works}))$
 $\cup (\text{works} - \sigma_{\text{company_name} = \text{'ABC company'}} (\text{works}))$

7) Differentiate between relational algebra and relational calculus? Define TRC and DRC?

Relational algebra	Relational calculus
It is a procedural language.	Relational Calculus is a Declarative(non-procedural) language.
Relational algebra means how to obtain the result.	Relational calculus means what result we have to obtain.
In relational algebra, the order is specified in which the operations have to be performed.	In relational calculus, the order is not specified.
Relational algebra is independent of domain.	Relational calculus can be domain dependent because of domain relational calculus.
The SQL includes only some features from the relational algebra.	SQL is based to a greater extent on the tuple relational calculus.

The evaluation of the query relies on the order specification in which the operations must be performed.	The order of operations does not matter in relational calculus for the evaluation of queries.
The basic operation included in relational algebra are: 1. Selection (σ) 2. Projection (Π) 3. Union ($\cup$) 4. Set Difference ($-$) 5. Cartesian product ($\times$) 6. Rename (ρ)	Relational Calculus is denoted as: $\{ t \mid P(t) \}$ Where, t: the set of tuples p: is the condition which is true for the given set of tuples.

Tuple Relational Calculus (TRC)

- ✓ It is a nonprocedural query language, where each query is of the form $\{t \mid P(t)\}$
- ✓ where t is a tuple variable and P(t) is a formula(similar to predicate calculus) that describes the conditions that the tuples in the result must satisfy .
- ✓ t is a *tuple variable*, $t[A]$ denotes the value of tuple t on attribute A
- ✓ $t \in r$ denotes that tuple t is in relation

Example:

Let us consider the following schema

Loan(loan_number,branch_name,amount)

Find the loan-number, branch-name, and amount for loans of over \$1200

$\{t \mid t \in loan \wedge t[amount] > 1200\}$

Domain Relational Calculus(DRC)

Domain Relational Calculus is a non-procedural query language equivalent in power to Tuple Relational Calculus. Domain Relational Calculus provides only the description of the query but it does not provide the methods to solve it.

Each query is an expression of the form:

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$$

where, $\langle x_1, x_2, x_3, \dots, x_n \rangle$ represents resulting domains variables and $P(x_1, x_2, x_3, \dots, x_n)$ represents the condition or formula equivalent to the Predicate calculus.

Example:

Let us consider the following schema

Loan(loan_number,branch_name,amount)

Find the loan-number, branch-name, and amount for loans of over \$1200

$$\{ \langle l, b, a \rangle \mid \langle l, b, a \rangle \in \text{loan} \wedge a > 1200 \}$$

8)

i) Employee(person_name,street,city)
Works(person_name,company_name,salary)
Company(company_name,city)

Write the relational algebra expression for the query “**Find the names of all employees whose company is located in pokhara**”. Construct the initial operator tree and final efficient operator tree after applying transformation rules.

Solution :

To Find the names of all employees whose company is located in pokhara from above schemas the relational algebraic expression can be written as

$$\Pi_{\text{person_name}}(\sigma_{\text{city}=\text{Pokhara}}(\text{Works} \bowtie \text{Company}))$$

In the above expression, the size of intermediate relation is large. Here, the tuples that are not required for final results are also participated in join operation. So we can apply some transformation rules to reduce the size of intermediate relation.

As we know, Natural joins are also commutative.

$$E1 \bowtie E2 \equiv E2 \bowtie E1$$

Now, above relational algebraic expression can be written as,

$$\Pi_{\text{person_name}}(\sigma_{\text{city}=\text{'Pokhara'}}(\text{Company} \bowtie \text{Works}))$$

By using distribution rule of selection ,

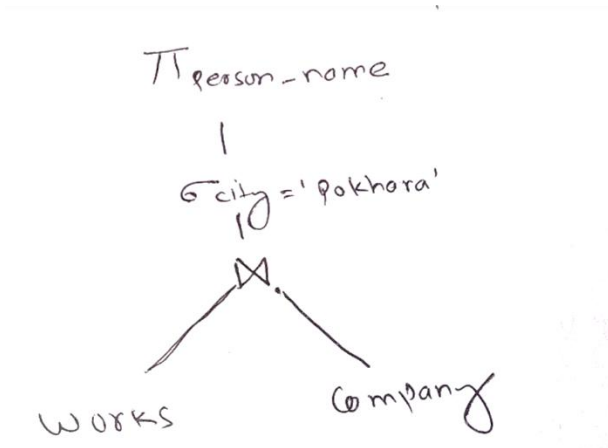
$$\sigma_{\theta_0}(E1 \bowtie E2) \equiv (\sigma_{\theta_0}(E1)) \bowtie E2$$

When all the attributes in θ_0 involve only the attributes of one of the expressions ($E1$) being joined.

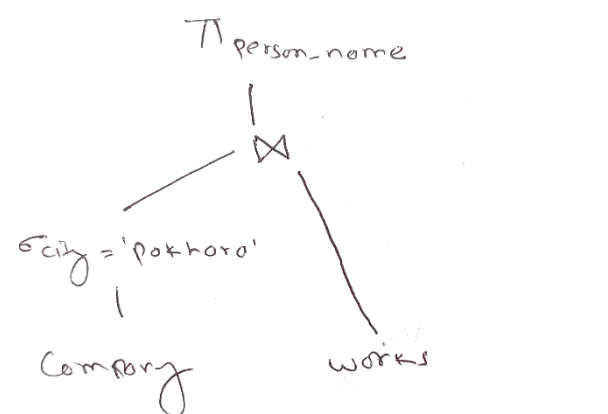
$$\Pi_{\text{person_name}}((\sigma_{\text{city}=\text{'Pokhara'}}(\text{Company})) \bowtie \text{Works})$$

Which is efficient form than the previous expression because performing early selection reduces the size of relation to be joined.

Initial Operator tree



Final operator tree after multiple transformations



ii) consider the following relational schema

Sailors (sid,sname,rating,age)

Boats(bid,bname,color)

Reserves(sid,bid,day)

Write a relational algebra expression for the query "find the name of sailors who have reserved red or green boat". Construct the initial operator tree and final efficient operator tree after applying transformation rules.

Solution :

To find the name of sailors who have reserved a red or green boat, the relational algebraic expression can be written as:

$$\Pi_{\text{name}} (\sigma_{\text{color}=\text{"red"} \vee \text{color}=\text{"green"}} (\text{Sailors} \bowtie (\text{Reserves} \bowtie \text{Boats})))$$

In the above expression, the size of intermediate relation is large. Here, the tuples that are not required for final results are also participated in join operation. So we can apply some transformation rules to reduce the size of intermediate relation.

As we know, Natural joins are associative,

$$(E1 \bowtie E2) \bowtie E3 \equiv E1 \bowtie (E2 \bowtie E3)$$

Now, above relational algebraic expression can be written as,

$$\Pi_{\text{name}} (\sigma_{\text{color}=\text{"red"} \vee \text{color}=\text{"green"}} ((\text{Sailors} \bowtie \text{Reserves}) \bowtie \text{Boats}))$$

As we know, Natural joins are also commutative.

$$E1 \bowtie E2 \equiv E2 \bowtie E1$$

Now, above relational algebraic expression can be written as,

$$\Pi_{\text{name}} (\sigma_{\text{color}=\text{"red"} \vee \text{color}=\text{"green"}} (\text{Boats} \bowtie (\text{Sailors} \bowtie \text{Reserves})))$$

By using distribution rule of selection ,

$$\sigma_{\theta_0} (E1 \bowtie E2) \equiv (\sigma_{\theta_0}(E1)) \bowtie E2$$

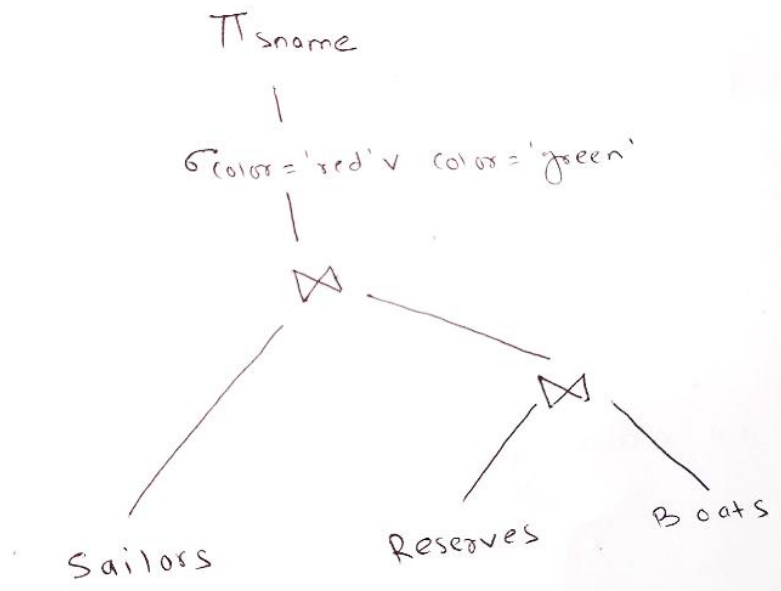
When all the attributes in θ_0 involve only the attributes of one of the expressions ($E1$) being joined.

Now, above expression can be written as:

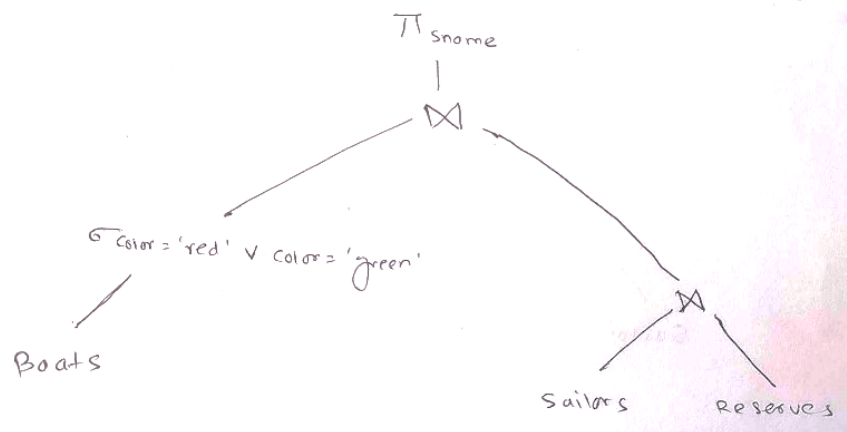
$$\Pi_{\text{name}} ((\sigma_{\text{color}=\text{"red"} \vee \text{color}=\text{"green"}} (\text{Boats})) \bowtie (\text{Sailors} \bowtie \text{Reserves}))$$

Which is efficient form than the previous expression because performing early selection reduces the size of relation to be joined.

Initial Operator tree



Final Efficient operator tree after applying multiple transformation.



- iii) Consider the relational schema
- Employee(person_name,street,city)
 - Works(person_name,company_name,salary)
 - Company(company_name,city)

Write the relational algebra expression for the query **“Find the names of all employees who lives in Pokhara”**. Construct the initial operator tree and final efficient operator tree after applying transformation rules.

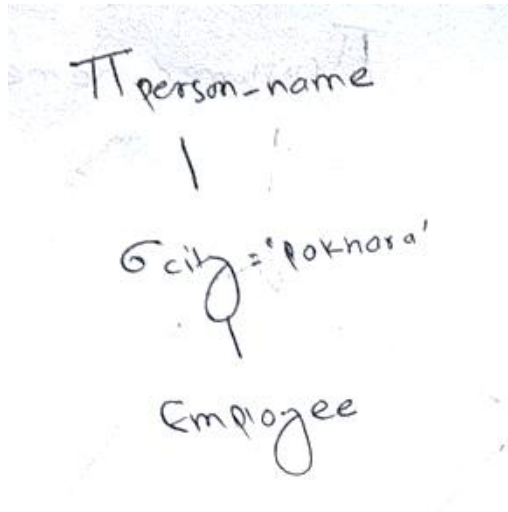
Solution :

To Find the names of all employees who lives in pokhara from above schemas the relational algebraic expression can be written as

$\Pi_{\text{person_name}}(\sigma_{\text{city}='Pokhara'}(\text{Employee}))$

Here, the above expression is already in its most efficient form. So the initial operator tree and final efficient operator tree are the same. Hence, we have

Initial /final operator tree



- iv) Make an operator tree for the following SQL expression.
- Select customer_name
From branch,account,depositor
Where branch_city='btl' AND balance>2000;

For the above SQL expression let us consider the following schemas

Branch(branch_name,branch_city,assets)

Account(account_number,branch_name,balance)

Depositor (customer_id,account_number)

Now, relational algebraic expression for the above SQL expression can be written as:

$\Pi_{\text{customer_name}} (\sigma_{\text{branch_city}='btl' \wedge \text{balance}>2000} (\text{branch} \bowtie (\text{account} \bowtie \text{depositor})))$

In the above expression, the size of intermediate relation is large. Here, the tuples that are not required for final results are also participated in join operation. So we can apply some transformation rules to reduce the size of intermediate relation.

As we know, Natural joins are associative,

$$(E1 \bowtie E2) \bowtie E3 \equiv E1 \bowtie (E2 \bowtie E3)$$

Now, above relational algebraic expression can be written as,

$$\Pi_{\text{customer_name}} (\sigma_{\text{branch_city}='btl' \wedge \text{balance}>2000} ((\text{branch} \bowtie \text{account}) \bowtie \text{depositor}))$$

By using distributive rule of selection,

$$\sigma_{\theta_1 \wedge \theta_2} (E1 \bowtie E2) \equiv (\sigma_{\theta_1}(E1)) \bowtie (\sigma_{\theta_2}(E2))$$

When θ_1 involves only the attributes of $E1$ and θ_2 involves only the attributes of $E2$.

Here, above subexpression becomes,

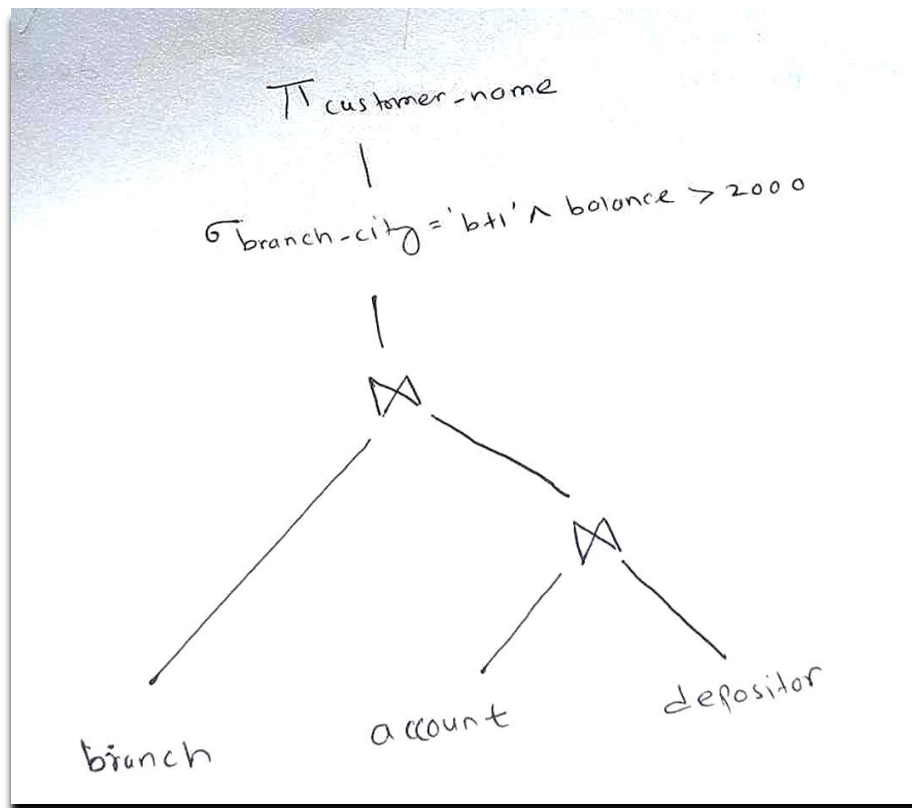
$$(\sigma_{\text{branch_city}='btl'}(\text{branch})) \bowtie (\sigma_{\text{balance}>2000}(\text{account}))$$

Now, finally relational algebraic expression becomes,

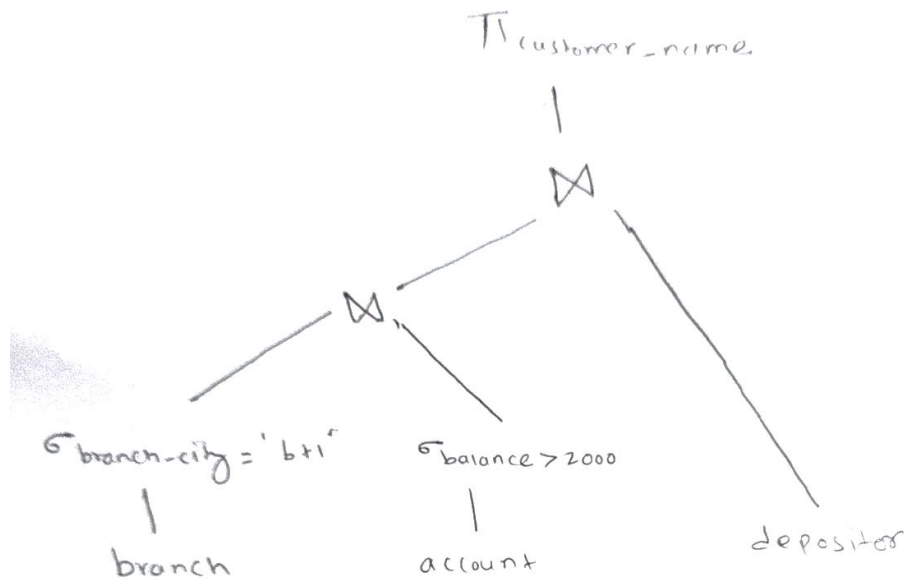
$$\Pi_{\text{customer_name}} (((\sigma_{\text{branch_city}='btl'}(\text{branch})) \bowtie (\sigma_{\text{balance}>2000}(\text{account}))) \bowtie \text{depositor})$$

Which is efficient form than the previous expression because performing early selection reduces the size of relation to be joined.

Initial Operator tree



Final Efficient operator tree after applying multiple transformation



- i) Find the closure set of attributes and functional dependency from following
 $A \rightarrow BC, B \rightarrow C, AB \rightarrow C, AC \rightarrow D$

The set of all those attributes which can be functionally determined from an attribute set is called as closure of that attribute set.

As we know,

$X^+ \rightarrow$ contains the set of attributes determined by X

Now,

$A^+ = \{ABCD\}$

(Here, Attribute A can determine A itself. A can determine BC from above functional dependency, again AC can determine D)

Similarly,

$B^+ = \{BC\}$

$C^+ = \{C\}$

$D^+ = \{D\}$

Closure of a functional dependency in database design refers to the set of functional dependencies that can be derived from a given set of functional dependencies.

Here, Given functional dependency,

$F = \{A \rightarrow BC, B \rightarrow C, AB \rightarrow C, AC \rightarrow D\}$

Now for F^+

$A \rightarrow B, A \rightarrow C$

[$\because A \rightarrow BC$ and applying Decomposition rule]

$A \rightarrow C$

[$\because A \rightarrow B, B \rightarrow C$ and applying Transitivity rule]

$AB \rightarrow D$

[$\because B \rightarrow C$ and $AC \rightarrow D$ and applying pseudo transitivity rule]

$AB \rightarrow CD$

[$\because AB \rightarrow C$ and $AB \rightarrow D$ and applying union rule]

Now,

$F^+ = \{A \rightarrow BC, B \rightarrow C, AB \rightarrow C, AC \rightarrow D, A \rightarrow B, A \rightarrow C, AB \rightarrow D, AB \rightarrow CD\}$

- ii) Let $R = \{A, B, C, D, E, F\}$ $F = \{A \rightarrow BC, E \rightarrow CF, B \rightarrow E, CD \rightarrow EF\}$ Now compute $(AB)^+$

Solution:

1st Iteration

Let result=AB then,

For $A \rightarrow BC$

$A \subseteq \{AB\}$ is true.

$\therefore \text{result} = \{ABC\}$

For $E \rightarrow CF$

$E \subseteq \{ABC\}$ is false .

$\therefore \text{result} = \{ABC\}$

For $B \rightarrow E$

$B \subseteq \{ABC\}$ is true.

$\therefore \text{result} = \{ABCE\}$

For $CD \rightarrow EF$

$CD \subseteq \{ABCE\}$ is false .

$\therefore \text{result} = \{ABCE\}$

2nd Iteration

For $A \rightarrow BC$

$A \subseteq \{ABCE\}$ is true.

$\therefore \text{result} = \{ABCE\}$

For $E \rightarrow CF$

$E \subseteq \{ABCE\}$ is True.

$\therefore \text{result} = \{ABCEF\}$

For $B \rightarrow E$

$B \subseteq \{ABCEF\}$ is true.

$\therefore \text{result} = \{ABCE\}$

For $CD \rightarrow EF$

$CD \subseteq \{ABCEF\}$ is false .

$\therefore \text{result} = \{ABCEF\}$

3rd Iteration

For $A \rightarrow BC$

$A \subseteq \{ABCEF\}$ is true.

$\therefore \text{result} = \{ABCEF\}$

For $E \rightarrow CF$

$E \subseteq \{ABCEF\}$ is True.

$\therefore \text{result} = \{ABCEF\}$

For $B \rightarrow E$

$B \subseteq \{ABCEF\}$ is true.

$\therefore \text{result} = \{ABCEF\}$

For $CD \rightarrow EF$

$CD \subseteq \{ABCEF\}$ is false .

$\therefore \text{result} = \{ABCEF\}$

Here ,result does not change further.

$(AB)^+ = \{ABCEF\}$

Procedure for determining the candidate key

Candidate Key:

An attribute or the combination of attributes is called candidate key if and only if:

- They derive all the attributes of the relation.
- They are the minimal subset of the super key.

Note:

- ✓ Candidate keys can be either simple or composite.
- ✓ Minimal subset is not with respect to the no of attributes however it always refer to the minimal level of subset which does not have any proper subsets that derives all the attributes of the relation.
- ✓ A relation can have more than one candidate key.

Determining candidate key

- ✓ Compute closure set of each attributes.
- ✓ Any attribute is called candidate key of any if attribute closure is equal to the relation.
 - Attribute that are part of candidate key are called **prime attribute**.
 - Attribute that are not part of candidate key are called **non-prime attribute**.

Example:

$R=(A,B,C,D)$

$F=\{AB \rightarrow C, C \rightarrow D, D \rightarrow A\}$

List all the candidate keys, prime and non-prime attributes.

Solution:

For A

$A^+ = \{A\}$; A is not candidate key

For B

$B^+ = \{B\}$; B is not candidate key

For C

$C^+ = \{CDA\}$; C is not candidate key

For D

$D^+ = \{DA\}$; D is not candidate key

For AB

$(AB)^+ = \{ABCD\}$; AB is candidate key

For AC

$(AC)^+ = \{ACD\}$; AC is not candidate key

For AD

$(AD)^+ = \{AD\}$; AD is not candidate key

For BC

$(BC)^+ = \{BCDA\}$; BC is a candidate key

For BD

$(BD)^+ = \{BDAC\}$; BD is a candidate key

For CD

$(CD)^+ = \{CDA\}$; CD is not a candidate key

$\therefore$ Candidate key = $\{AB, BC, BD\}$

$\therefore$ prime attribute = $\{A, B, C, D\}$

Here is no any non-prime attribute.

11. Create a un-normalized table and normalize it up to 3NF.

Let us consider the following table.

Student_id	Student_name	Salutation	Subject_id	Subject_name	Marks	contact
1	Ram	Mr.	1	DBMS	80	9851155666, 9802466774
2	Sita	Ms.	1	DBMS	70	9847025512, 9818065256
1	Ram	Mr.	2	C++	75	9851155666, 9802466774
3	Gita	Ms.	2	C++	90	9851167343

For 1 NF

✓ A relation is in 1NF if all the contained values should be atomic values.

But the above table is not in 1NF because there are multiple values in column named contact, which violates the rule of 1NF.

Now above table can be converted into 1NF as follows.

Student

Student_id	Student_name	Salutation	Subject_id	Subject_name	Marks	Contact
1	Ram	Mr.	1	DBMS	80	9851155666, 9802466774
2	Sita	Ms.	1	DBMS	70	9847025512, 9818065256
1	Ram	Mr.	2	C++	75	9851155666, 9802466774
3	Gita	Ms.	2	C++	90	9851167343

Student_contact

Student_id	Contact
1	9851155666
1	9802466774
2	9847025512
2	9818065256
3	9851167343

For 2 NF

A relation will be in 2NF if

- ✓ It is in 1NF and
- ✓ All non-prime attributes are fully functional dependent on the key attribute or primary key
(There must not be any partial dependency)

In the above table named **Student** {Student_id +Subject_id} can be considered as composite primary key. Due to this combination, we can uniquely identify all the records of the table.

But here partial dependency exists. All attributes are not fully functional dependent on primary key.

For example:

Student_id → Student_name

Subject_id → Subject_name

Now, we can remove this partial dependency as follows.

Student_info

Student_id	Student_name	Salutation
1	Ram	Mr.
2	Sita	Ms.
3	Gita	Ms.

Subject_info

Subject_id	Subject_name
1	DBMS
2	C++

Student_contact

Student_id	Contact
1	9851155666
1	9802466774
2	9847025512
2	9818065256
3	9851167343

For 3NF

For any relation to be in 3NF it must satisfied following properties.

- ✓ It is in 2NF.
- ✓ Every non-prime attribute is non-transitively dependent on the primary key. Which means there should not be case that non-prime attribute is functionally dependent on another non-prime attribute.

In above table named **Student_info**

Finding functional dependencies

Student_id $\rightarrow$ Student_name

Student_name $\rightarrow$ Salutation.

Here we can find transitive dependencies. Salutation is transitively dependent on primary key SID. Now, we can remove this dependency as follows.

Student_info

Student_id	Student_name	Salutation
1	Ram	Mr.
2	Sita	Ms.
3	Gita	Ms.

Saluation_info

Student_name	Salutation
Ram	Mr.
Sita	Ms.
Gita	Ms.

Subject_info

Subject_id	Subject_name
1	DBMS
2	C++

Student_contact

Student_id	Contact
1	9851155666
1	9802466774
2	9847025512
2	9818065256
3	9851167343

Now, we can see that above tables satisfy the condition upto 3NF.

12.

- i) Consider the following table. Identify all the functional dependencies in the table and convert it into 3NF.

Employee

EmpNo	EName	City	Post	Salary
E001	Ram	Pokhara,Dharan	Manager	30000
E002	Sita	Kathmandu	Analyst	25000
E003	Bharat	Bharatpur,Pokhara	Programmer	22000
E004	Laxman	Butwal	Manager	30000
E005	Dashrath	Lalitpur,Kathmandu	Analyst	25000

From the above table we can find the following functional dependencies.

$\text{EmpNo} \rightarrow \{\text{EName}, \text{City}, \text{Post}, \text{Salary}\}$

$\text{Post} \rightarrow \text{Salary}$

To convert the given table into 3NF, Lets check for 1NF

For table to be in 1NF, Each cell must be single valued i.e no repeating groups.

But, the above table violates the rule of 1NF. Now converting in 1NF we get

Employee_info

EmpNo	EName	Post	Salary
E001	Ram	Manager	30000
E002	Sita	Analyst	25000
E003	Bharat	Programmer	22000
E004	Laxman	Butwal	30000
E005	Dashrath	Analyst	25000

City_info

EmpNo	City
E001	Pokhara
E001	Dharan
E002	Kathmandu
E003	Bhaktapur
E003	Pokhara
E004	Manager
E005	Lalitpur
E005	Kathmandu

For a relation to be in 2NF,

1. It must already be in 1NF
2. Every non-prime attribute is fully dependent on primary key (has no partial dependency)

Here ,In above relation Employee_info EmpNo can be considered as primary key. Here EmpNo determine all other attributes. So there is no partial dependency. So it is in 2NF.

For 3NF,

1. It must already be in 2NF
2. There should not be transitive dependency. (i.e. there should not be the case that non-prime attribute is functionally dependent on another non-prime attribute)

Here, In the table Employee_info Salary is determined by Post which is non-prime attribute. We can convert above relation Employee_info into 3NF as follows and finally relation becomes.

Emp_post

EmpNo	Ename	Post
E001	Ram	Manager
E002	Sita	Analyst
E003	Bharat	Programmer
E004	Laxman	Butwal
E005	Dashrath	Analyst

Emp_salary

Post	Salary
Manager	30000
Analyst	25000
Programmer	22000

City_info

EmpNo	City
E001	Pokhara
E001	Dharan
E002	Kathmandu
E003	Bhaktapur
E003	Pokhara
E004	Manager
E005	Lalitpur
E005	Kathmandu

ii) Explain second Normal form? Whether the following table is in 2NF?

<u>Sector</u>	<u>PlotNo</u>	City
A	1	KTM
A	2	KTM
A	3	KTM
B	1	Pokhara
B	2	Pokhara

Explain the different anomalies in the database table.

Solution:

In the following table we can find following anomalies.

Insertion Anomaly: Insertion anomalies occur when it is not possible to add new data to table without including additional or unrelated information(i.e. data cannot be added without providing information of all attributes.

In the above table if we want to insert new row for different sector or city ,we must also provide valid plot number.

Update Anomaly: If the table if we want to update the city for particular sector, we must have to ensure that updation must be done in all rows in which particular sector is exist, otherwise it leads to inconsistency of database.

Deletion Anomaly: If we want to delete a row corresponding to specific sector and plot number, we might unintentionally delete information about city.

A relation is in 2NF if and only if

1. It is in 1NF and
2. Every non-prime attribute is fully dependent on primary key.(has no partial dependency)

Here ,considering (sector +plot No) as composite primary key as per question, there exist a partial dependency ,because city is functionally dependent on sector.

sector $\rightarrow$ city

so it is not in 2NF.

By removing partial dependencies ,this table can be converted to 2NF as follows:

Sector_info

Sector	City
A	KTM
B	Pokhara

Plot_info

Sector	Plot_no
A	1
A	2
A	3
B	1
B	2

- iii) What do you mean by functional dependency, multivalued dependency and transitive dependency in Normalization process of Database? Why Normalization is needed? Assume the un-normalized relation as given below and find the final normalized logical ER diagram normalizing the un-normalized relation upto 3NF explaining what you going to check at each normal step.

Roll.No	Name	SubID	SubName	FeePaid
1	Hari Man Dangol	DBMS	Database Management System	20000
2	Mohan Prasad Shah	DBMS	Database Management System	20000
3	Indira Rimal	DBMS	Database Management System	30000
1	Hari Man Dangol	CPROG	C programming	20000
2	Mohan Prasad Shah	CPROG	C programming	15000
3	Indira Rimal	MATH	Mathematics	30000

(For functional dependency ,multivalued dependency and transitive dependency refer note)

As we know, Database Normalization is a technique of organizing the data in the database. It is a systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics like Insertion, Update and Deletion Anomalies.

Normalization is needed due to the following reasons:

- ✓ Normalization helps in reducing redundancy by organizing data into separate tables and ensuring that each piece of data is stored only once.
- ✓ It optimizes data storage, reducing disk space usage and storage costs.
- ✓ Normalization helps maintain data integrity by reducing the risk of anomalies such as insertion, update, and deletion anomalies.
- ✓ Well-normalized databases typically perform better in terms of query execution and data retrieval.
- ✓ Normalized databases are generally more scalable and adaptable to changing requirements.

To convert the given Un-normalized relation into 3NF,

Let's check for 1NF.

- ✓ For relation to be in 1NF, each cell must be single valued i.e. no repeating groups.

But in the above given relation there is no repeating groups. So the given relation is already in 1NF.

For a relation to be in 2NF,

1. It must already be in 1NF
2. Every non-prime attribute is fully dependent on primary key (has no partial dependency)

Here, in above relation (Roll.No +SubID) is composite primary key.

But there exists a partial dependency such as

Roll.No $\rightarrow$ Name

SubID $\rightarrow$ SubName

By removing partial dependencies, this table can be converted to 2NF as follows:

Student_info

Roll.No	Name
1	Hari Man Dangol
2	Mohan Prasad Shah
3	Indira Rimal

Subject_info

SubID	SubName
DBMS	Database Management System
CPROG	C programming
MATH	Mathematics

Fee_info

Roll.No	SubID	FeePaid
1	DBMS	20000
2	DBMS	20000
3	DBMS	30000
1	CPROG	20000
2	CPROG	15000
3	MATH	30000

For 3NF,

1. It must already be in 2NF
2. There should not be transitive dependency. (i.e. there should not be the case that non-prime attribute is functionally dependent on another non-prime attribute)

In above 2NF relations there is no any transitive dependency. So its is already in 3NF.